sponsors 29 patreon 122 liberapay 5 paypal 297

New! Render PlantUML diagrams directly inside GitHub with our official browser extension — No server. No tokens. No tracking. Zero permissions but clipboard. — Try it out and let us know what you think! Add to Chrome Add to Firefox

Activity diagram

Remove Gone in a touch with Generative Remove. Learn more Adobe Lightroom

An activity diagram describes a workflow or process: the steps, the order they happen in, and the decisions and parallel paths along the way. It's well suited to business processes, algorithms, and anything that flows from a start to an end.

In PlantUML, you write each activity on its own line, using arrows and keywords like `if`, `repeat`, and `fork` to express control flow. The diagram is generated for you.

- **Text in, diagram out.** Describe the flow top to bottom, branches and loops included.
- **Reads top to bottom.** The order in your text is the order in your diagram.
- **Easy to restructure.** Reordering steps or adding a branch is a text edit, not a redraw.

If you're maintaining older diagrams, the previous syntax is still supported but no longer recommended for new diagrams.

See common commands for features available across all diagram types.

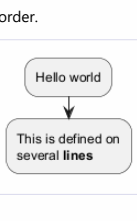
Simple action

Activities label starts with `:` and ends with `;`.

Text formatting can be done using creole wiki syntax.

They are implicitly linked in their definition order.

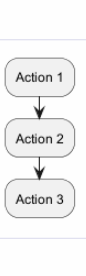
```
@startuml
:Hello world;
:This is defined on
several **lines**;
@enduml
```



Other simple action *(defined as a list)*

Simple action list separated by `-`

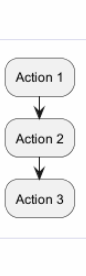
```
@startuml
- Action 1
- Action 2
- Action 3
@enduml
```



Simple action list separated by `*`

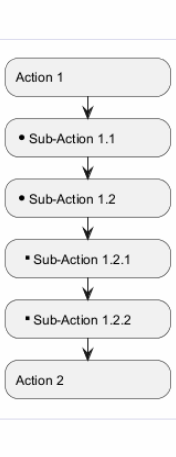
With one level

```
@startuml
* Action 1
* Action 2
* Action 3
@enduml
```



With several levels

```
@startuml
<style>
element {MinimumWidth 150}
</style>
* Action 1
** Sub-Action 1.1
** Sub-Action 1.2
*** Sub-Action 1.2.1
*** Sub-Action 1.2.2
* Action 2
@enduml
```

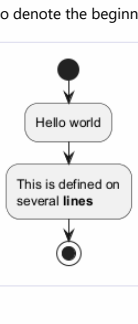


[Ref. GH-2376]

Start/Stop/End

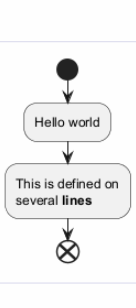
You can use `start` and `stop` keywords to denote the beginning and the end of a diagram.

```
@startuml
start
:Hello world;
:This is defined on
several **lines**;
stop
@enduml
```



You can also use the `end` keyword.

```
@startuml
start
:Hello world;
:This is defined on
several **lines**;
end
@enduml
```



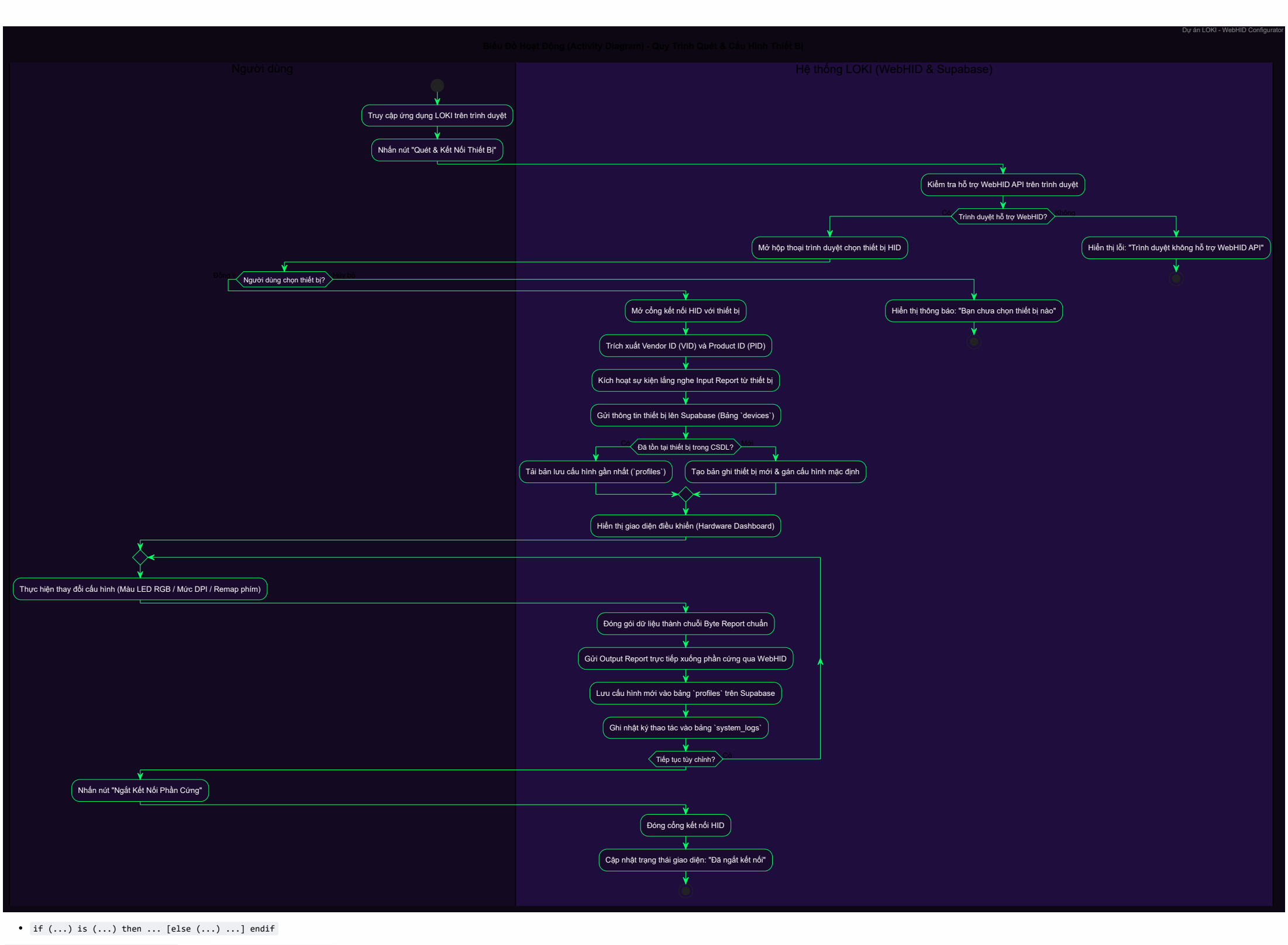
Conditional [if, then, else, endif]

You can use `if`, `then`, `else` and `endif` keywords to put tests in your diagram. Labels can be provided using parentheses.

The 3 syntaxes are possible:

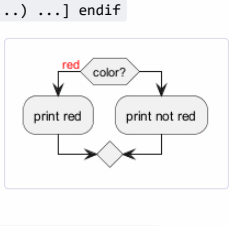
- `if (...) then (...) ... [else (...) ...] endif`

```
@startuml LOKI_Activity_Diagram
skinparam backgroundColor #0D0714
skinparam ActivityBackgroundColor #1A0B2E
skinparam ActivityBorderColor #00FF66
skinparam ActivityFontColor #FFFFFF
skinparam ActivityFontName Arial
skinparam ArrowColor #00FF66
skinparam DiamondBackgroundColor #1A0B2E
skinparam DiamondBorderColor #00FF66
skinparam DiamondFontColor #FFFFFF

header Dự án LOKI - WebHID Configurator
title Biểu Đồ Hoạt Động (Activity Diagram) - Quy Trình Quét & Cấu Hình Thiết Bị

|#130A21|Người dùng|
start
:Truy cập ứng dụng LOKI trên trình duyệt;
:Nhấn nút "Quét & Kết Nối Thiết Bị";

|#1F0E3D|Hệ thống LOKI (WebHID & Supabase)|
:Kiểm tra hỗ trợ WebHID API trên trình duyệt;

if (Trình duyệt hỗ trợ WebHID?) then (Có)
  :Mở hộp thoại trình duyệt chọn thiết bị HID;

  |Người dùng|
  if (Người dùng chọn thiết bị?) then (Đồng ý)
    |Hệ thống LOKI (WebHID & Supabase)|
    :Mở cổng kết nối HID với thiết bị;
    :Trích xuất Vendor ID (VID) và Product ID (PID);
    :Kích hoạt sự kiện lắng nghe Input Report từ thiết bị;

    :Gửi thông tin thiết bị lên Supabase (Bảng `devices`);

    if (Đã tồn tại thiết bị trong CSDL?) then (Có)
      :Tải bản lưu cấu hình gần nhất (`profiles`);
    else (Mới)
      :Tạo bản ghi thiết bị mới & gán cấu hình mặc định;
    endif

    :Hiển thị giao diện điều khiển (Hardware Dashboard);

    |Người dùng|
    repeat
      :Thực hiện thay đổi cấu hình (Màu LED RGB / Mức DPI / Remap phím);
      |Hệ thống LOKI (WebHID & Supabase)|
      :Đóng gói dữ liệu thành chuỗi Byte Report chuẩn;
      :Gửi Output Report trực tiếp xuống phần cứng qua WebHID;
      :Lưu cấu hình mới vào bảng `profiles` trên Supabase;
      :Ghi nhật ký thao tác vào bảng `system_logs`;
    repeat while (Tiếp tục tùy chỉnh?) is (Có)

    |Người dùng|
    :Nhấn nút "Ngắt Kết Nối Phần Cứng";

    |Hệ thống LOKI (WebHID & Supabase)|
    :Đóng cổng kết nối HID;
    :Cập nhật trạng thái giao diện: "Đã ngắt kết nối";
    stop

  else (Hủy bỏ)
    |Hệ thống LOKI (WebHID & Supabase)|
    :Hiển thị thông báo: "Bạn chưa chọn thiết bị nào";
    stop
  endif

else (Không)
  :Hiển thị lỗi: "Trình duyệt không hỗ trợ WebHID API";
  stop
endif

@enduml
```


Người dùng
Truy cập ứng dụng LOKI trên trình duyệt
Nhấn nút "Quét & Kết Nối Thiết Bị"
Kiểm tra hỗ trợ WebHID API trên trình duyệt
Trình duyệt hỗ trợ WebHID?
Mở hộp thoại trình duyệt chọn thiết bị HID
Hiển thị lỗi: "Trình duyệt không hỗ trợ WebHID API"
Người dùng chọn thiết bị?
Mở cổng kết nối HID với thiết bị
Hiển thị thông báo: "Bạn chưa chọn thiết bị nào"
Trích xuất Vendor ID (VID) và Product ID (PID)
Kích hoạt sự kiện lắng nghe Input Report từ thiết bị
Gửi thông tin thiết bị lên Supabase (Bảng `devices`)
Đã tồn tại thiết bị trong CSDL?
Tải bản lưu cấu hình gần nhất (`profiles`)
Tạo bản ghi thiết bị mới & gán cấu hình mặc định
Hiển thị giao diện điều khiển (Hardware Dashboard)
Thực hiện thay đổi cấu hình (Màu LED RGB / Mức DPI / Remap phím)
Đóng gói dữ liệu thành chuỗi Byte Report chuẩn
Gửi Output Report trực tiếp xuống phần cứng qua WebHID
Lưu cấu hình mới vào bảng `profiles` trên Supabase
Ghi nhật ký thao tác vào bảng `system_logs`
Tiếp tục tùy chỉnh?
Nhấn nút "Ngắt Kết Nối Phần Cứng"
Đóng cổng kết nối HID
Cập nhật trạng thái giao diện: "Đã ngắt kết nối"

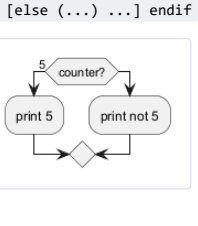
- `if (...) is (...) then ... [else (...) ...] endif`

```
@startuml
if (color?) is (<color:red>red) then
:print red;
else
:print not red;
endif
@enduml
```



- `if (...) equals (...) then ... [else (...) ...] endif`

```
@startuml
if (counter?) equals (5) then
:print 5;
else
:print not 5;
endif
@enduml
```

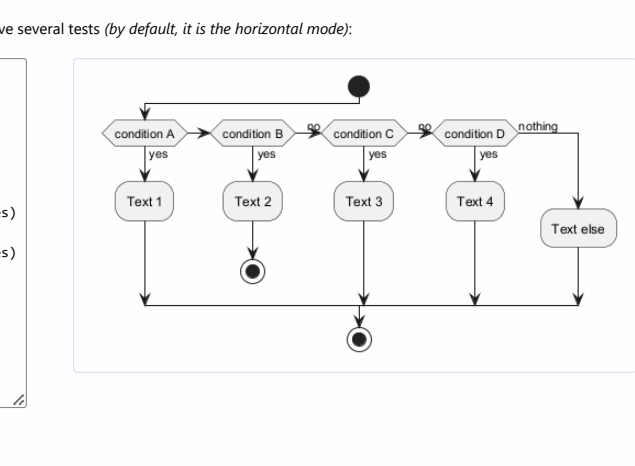


[Ref. QA-301]

Several tests (horizontal mode)

You can use the `elseif` keyword to have several tests *(by default, it is the horizontal mode)*:

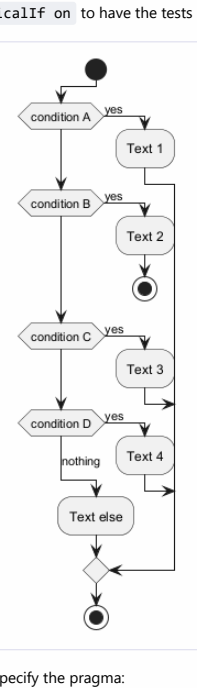
```
@startuml
start
if (condition A) then (yes)
  :Text 1;
elseif (condition B) then (yes)
  :Text 2;
  stop
(no) elseif (condition C) then (yes)
  :Text 3;
(no) elseif (condition D) then (yes)
  :Text 4;
else (nothing)
  :Text else;
endif
stop
@enduml
```



Several tests (vertical mode)

You can use the command `!pragma useVerticalIf on` to have the tests in vertical mode:

```
@startuml
!pragma useVerticalIf on
start
if (condition A) then (yes)
  :Text 1;
elseif (condition B) then (yes)
  :Text 2;
  stop
elseif (condition C) then (yes)
  :Text 3;
elseif (condition D) then (yes)
  :Text 4;
else (nothing)
  :Text else;
endif
stop
@enduml
```



You can use the `-P` command-line option to specify the pragma:

```
java -jar plantuml.jar -PuseVerticalIf=on
```

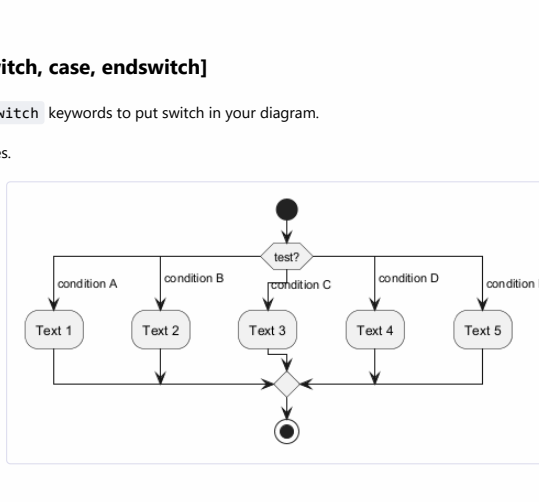
[Refs. QA-3931, GH-582]

Switch and case [switch, case, endswitch]

You can use `switch`, `case` and `endswitch` keywords to put switch in your diagram.

Labels can be provided using parentheses.

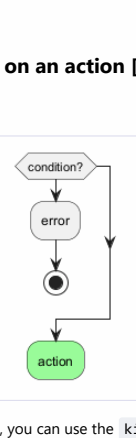
```
@startuml
start
switch (test?)
case ( condition A )
  :Text 1;
case ( condition B )
  :Text 2;
case ( condition C )
  :Text 3;
case ( condition D )
  :Text 4;
case ( condition E )
  :Text 5;
endswitch
stop
@enduml
```



Conditional with stop on an action [kill, detach]

You can stop action on a if loop.

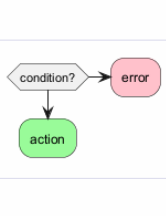
```
@startuml
if (condition?) then
  :error;
  stop
endif
:action; <<#palegreen>>
@enduml
```



But if you want to stop at the precise action, you can use the `kill` or `detach` keyword:

- `kill`

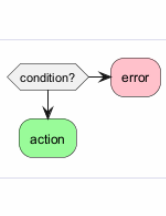
```
@startuml
if (condition?) then
  :error; <<#pink>>
  kill
endif
:action; <<#palegreen>>
@enduml
```



[Ref. QA-265]

- `detach`

```
@startuml
if (condition?) then
  :error; <<#pink>>
  detach
endif
:action; <<#palegreen>>
@enduml
```

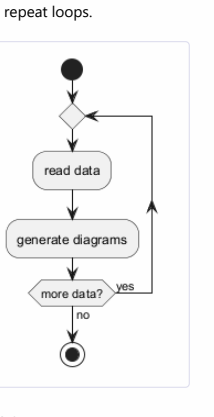


Repeat loop

Simple repeat loop

You can use `repeat` and `repeat while` keywords to have repeat loops.

```
@startuml

start

repeat
  :read data;
  :generate diagrams;
repeat while (more data?) is (yes) not (no)

stop

@enduml
```



Repeat loop with repeat action and backward action

It is also possible to use a full action as `repeat` target and insert an action in the return path using the `backward` keyword.

```
@startuml

start

repeat :foo as starting label;
  :read data;
  :generate diagrams;
backward:This is backward;
repeat while (more data?) is (yes)
->no;

stop

@enduml
```

[Ref. QA-5826]

Break on a repeat loop [break]

You can use the `break` keyword after an action on a loop.

```
@startuml
start
repeat
  :Test something;
    if (Something went wrong?) then (no)
      :OK; <<#palegreen>>
      break
    endif
    ->NOK;
    :Alert "Error with long text";
repeat while (Something went wrong with long text?) is (yes) not (no)
->//merged step//;
:Alert "Success";
stop
@enduml
```

[Ref. QA-6105]

Goto and Label Processing [label, goto]

⚠ It is currently only experimental 🚧

You can use `label` and `goto` keywords to denote goto processing, with:

- `label <label_name>`
- `goto <label_name>`

```
@startuml
title Point two queries to same activity\nwith `goto`
start
if (Test Question?) then (yes)
'space label only for alignment
label sp_lab0
label sp_lab1
'real label
label lab
:shared;
else (no)
if (Second Test Question?) then (yes)
label sp_lab2
goto sp_lab1
else
:nonShared;
endif
endif
:merge;
@enduml
```

[Ref. QA-15026, QA-12526 and initially QA-1626]

While loop

Simple while loop

You can use `while` and `endwhile` keywords to have while loop.

```
@startuml

start

while (data available?)
  :read data;
  :generate diagrams;
endwhile

stop

@enduml
```

It is possible to provide a label after the `endwhile` keyword, or using the `is` keyword.

```
@startuml
while (check filesize ?) is (not empty)
  :read file;
endwhile (empty)
:close file;
@enduml
```

While loop with backward action

It is also possible to insert an action in the return path using the `backward` keyword.

```
@startuml
while (check filesize ?) is (not empty)
  :read file;
  backward:log;
endwhile (empty)
:close file;
@enduml
```

[Ref. QA-11144]

Infinite while loop

If you are using `detach` to form an infinite while loop, then you will want to also hide the partial arrow that results using `-[hidden]->`

```
@startuml
:Step 1;
if (condition1) then
  while (loop forever)
   :Step 2;
  endwhile
  -[hidden]->
  detach
else
  :end normally;
  stop
endif
@enduml
```

Parallel processing [fork, fork again, end fork, end merge]

You can use `fork`, `fork again` and `end fork` or `end merge` keywords to denote parallel processing.

Simple `fork`

```
@startuml
start
fork
  :action 1;
fork again
  :action 2;
end fork
stop
@enduml
```

`fork` with end merge

```
@startuml
start
fork
  :action 1;
fork again
  :action 2;
end merge
stop
@enduml
```

[Ref. QA-5320]

```
@startuml
start
fork
  :action 1;
fork again
  :action 2;
fork again
  :action 3;
fork again
  :action 4;
end merge
stop
@enduml
```

```
@startuml
start
fork
  :action 1;
fork again
  :action 2;
  end
end merge
stop
@enduml
```

[Ref. QA-13731]

Label on `end fork` (or UML joinspec):

```
@startuml
start
fork
  :action A;
fork again
  :action B;
end fork {or}
stop
@enduml
```

```
@startuml
start
fork
  :action A;
fork again
  :action B;
end fork {and}
stop
@enduml
```

[Ref. QA-5346]

Other example

```
@startuml

start

if (multiprocessor?) then (yes)
  fork
    :Treatment 1;
  fork again
    :Treatment 2;
  end fork
else (monoproc)
  :Treatment 1;
  :Treatment 2;
endif

@enduml
```

Split processing

Split

You can use `split`, `split again` and `end split` keywords to denote split processing.

```
@startuml
start
split
   :A;
split again
   :B;
split again
   :C;
split again
   :a;
   :b;
end split
:D;
end
@enduml
```

Input split (multi-start)

You can use `hidden` arrows to make an input split (multi-start):

```
@startuml
split
   -[hidden]->
   :A;
split again
   -[hidden]->
   :B;
split again
   -[hidden]->
   :C;
end split
:D;
@enduml
```

```
@startuml
split
   -[hidden]->
   :A;
split again
   -[hidden]->
   :a;
   :b;
split again
   -[hidden]->
   (Z)
end split
:D;
@enduml
```

[Ref. QA-8662]

Output split (multi-end)

You can use `kill` or `detach` to make an output split (multi-end):

```
@startuml
start
split
   :A;
   kill
split again
   :B;
   detach
split again
   :C;
   kill
end split
@enduml
```

```
@startuml
start
split
   :A;
   kill
split again
   :b;
   :c;
   detach
split again
   (Z)
   detach
split again
   end
split again
   stop
end split
@enduml
```

Notes

Text formatting can be done using creole wiki syntax.

A note can be floating, using `floating` keyword.

```
@startuml

start
:foo1;
floating note left: This is a note
:foo2;
note right
  This note is on several
  //lines// and can
  contain <b>HTML</b>
  ====
  * Calling the method ""foo()"" is prohibited
end note
stop

@enduml
```

You can add note on backward activity:

```
@startuml
start
repeat :Enter data;
:Submit;
backward :Warning;
note right: Note
repeat while (Valid?) is (No) not (Yes)
stop
@enduml
```

[Ref. QA-11788]

You can add note on partition activity:

```
@startuml
start
partition "**process** HelloWorld" {
    note
        This is my note
        ----
        //Creole test//
    end note
    :Ready;
    :HelloWorld(i); <<output>>
    :Hello-Sent;
}
@enduml
```

[Ref. QA-2398]

Colors

You can specify a color for some activities.

```
@startuml

start
:starting progress;
:reading configuration files
These files should be edited at this point!; <<#HotPink>>
:ending of the process; <<#AAAAAA>>

@enduml
```

You can also use gradient color.

```
@startuml
start
partition #red/white testPartition {
        :testActivity; <<#blue\green>>
}
@enduml
```

[Ref. QA-4906]

Lines without arrows

You can use `skinparam ArrowHeadColor none` in order to connect activities using lines only, without arrows.

```
@startuml
skinparam ArrowHeadColor none
start
:Hello world;
:This is on defined on
several **lines**;
stop
@enduml
```

```
@startuml
skinparam ArrowHeadColor none
start
repeat :Enter data;
:Submit;
backward :Warning;
repeat while (Valid?) is (No) not (Yes)
stop
@enduml
```

Arrows

Using the `->` notation, you can add texts to arrow, and change their color.

It's also possible to have dotted, dashed, bold or hidden arrows.

```
@startuml
:foo1;
-> You can put text on arrows;
if (test) then
  -[#blue]->
  :foo2;
  -[#green,dashed]-> The text can
  also be on several lines
  and **very** long...;
  :foo3;
else
  -[#black,dotted]->
  :foo4;
endif
-[#gray,bold]->
:foo5;
@enduml
```

Simple colored arrow [link]

You can use simple colored arrow with the `link` keyword.

```
@startuml
:a;
link #blue
:b;
@enduml
```

Multiple colored arrow

You can use multiple colored arrow.

```
@startuml
skinparam colorArrowSeparationSpace 1
start
-[#red;#green;#orange;#blue]->
if(a?)then(yes)
-[#red]->
:activity;
-[#red]->
if(c?)then(yes)
-[#maroon,dashed]->
else(no)
-[#red]->
if(b?)then(yes)
-[#maroon,dashed]->
else(no)
-[#blue,dashed;dotted]->
:do a;
-[#red]->
:do b;
-[#red]->
endif
-[#red;#maroon,dashed]->
endif
-[#red;#maroon,dashed]->
elseif(e?)then(yes)
-[#green]->
if(c?)then(yes)
-[#maroon,dashed]->
else(no)
-[#green]->
if(d?)then(yes)
-[#maroon,dashed]->
else(no)
-[#green]->
:do something; <<continuous>>
-[#green]->
endif
-[#green;#maroon,dashed]->
partition dummy {
:some function;
}
-[#green;#maroon,dashed]->
endif
-[#green;#maroon,dashed]->

elseif(f?)then(yes)
-[#orange]->
:activity; <<continuous>>
-[#orange]->
else(no)
-[#blue,dashed;dotted]->
endif
stop
@enduml
```

[Ref. QA-4411]

Connector (or Circle)

You can use parentheses to denote connector.

```
@startuml
start
:Some activity;
(A)
detach
(A)
:Other activity;
@enduml
```

Color on connector

You can add color on connector.

```
@startuml
start
:The connector below
wishes he was blue;
#blue:(B)
:This next connector
feels that she would
be better off green;
#green:(G)
stop
@enduml
```

[Ref. QA-10077]

And even use style on Circle:

```
@startuml
<style>
circle {
  Backgroundcolor palegreen
  LineColor green
  LineThickness 2
}
</style>

(1)
:a;
(A)
@enduml
```

[Ref. QA-19975]

Grouping or partition

Group

You can group activity together by defining group:

```
@startuml
start
group Initialization {
    :read config file;
    :init internal variable;
}
group Running group {
    :wait for user interaction;
    :print information;
}

stop
@enduml
```

Partition

You can group activity together by defining partition:

```
@startuml
start
partition Initialization {
    :read config file;
    :init internal variable;
}
partition Running {
    :wait for user interaction;
    :print information;
}

stop
@enduml
```

It's also possible to change partition color:

```
@startuml
start
partition #lightGreen "Input Interface" {
    :read config file;
    :init internal variable;
}
partition Running {
    :wait for user interaction;
    :print information;
}
stop
@enduml
```

[Ref. QA-2793]

It's also possible to add link to partition:

```
@startuml
start
partition "[[http://plantuml.com partition_name]]" {
    :read doc. on [[http://plantuml.com plantuml_website]];
    :test diagram;
}
end
@enduml
```

[Ref. QA-542]

Group, Partition, Package, Rectangle or Card

You can group activity together by defining:

- group;
- partition;
- package;
- rectangle;
- card.

```
@startuml
start
group Group {
  :Activity;
}
floating note: Note on Group

partition Partition {
  :Activity;
}
floating note: Note on Partition

package Package {
  :Activity;
}
floating note: Note on Package

rectangle Rectangle {
  :Activity;
}
floating note: Note on Rectangle

card Card {
  :Activity;
}
floating note: Note on Card
end
@enduml
```

Swimlanes

Using pipe `|`, you can define swimlanes.

It's also possible to change swimlanes color.

```
@startuml
|Swimlane1|
start
:foo1;
|#AntiqueWhite|Swimlane2|
:foo2;
:foo3;
|Swimlane1|
:foo4;
|Swimlane2|
:foo5;
stop
@enduml
```

You can add `if` conditional or `repeat` or `while` loop within swimlanes.

```
@startuml
|#pink|Actor_For_red|
start
if (color?) is (red) then
:**action red**; <<#pink>>
:foo1;
else (not red)
|#lightgray|Actor_For_no_red|
:**action not red**; <<#lightgray>>
:foo2;
endif
|Next_Actor|
:foo3; <<#lightblue>>
:foo4;
|Final_Actor|
:foo5; <<#palegreen>>
stop
@enduml
```

You can also use `alias` with swimlanes, with this syntax:

- `|[#<color>|]<swimlane_alias>| <swimlane_title>`

```
@startuml
|#palegreen|f| fisherman
|c| cook
|#gold|e| eater
|f|
start
:go fish;
|c|
:fry fish;
|e|
:eat fish;
stop
@enduml
```

[Ref. QA-2681]

Detach or kill [detach, kill]

It's possible to remove an arrow using the `detach` or `kill` keyword.

- `detach`

```
@startuml
 :start;
 fork
   :foo1;
   :foo2;
 fork again
   :foo3;
   detach
 endfork
 if (foo4) then
   :foo5;
   detach
 endif
 :foo6;
 detach
 :foo7;
 stop
@enduml
```

- `kill`

```
@startuml
 :start;
 fork
   :foo1;
   :foo2;
 fork again
```

Simple action
Other simple action __(defined as a list)__
Start/Stop/End
Conditional [if, then, else, endif]
Switch and case [switch, case, endswitch]
Conditional with stop on an action [kill, detach]
Repeat loop
Break on a repeat loop [break]
Goto and Label Processing [label, goto]
While loop
Parallel processing [fork, fork again, end fork, end merge]
Split processing
Notes
Colors
Lines without arrows
Arrows
Simple colored arrow [link]
Multiple colored arrow
Connector (or Circle)
Color on connector
Grouping or partition
Swimlanes
Detach or kill [detach, kill]
Emoji as action (with `+icon+` stereotype)
SDL (Specification and Description Language) (with SDL sterotype)
UML (Unified Modeling Language) Shape (with UML stereotype)
Complete example
Condition Style
Condition End Style
Using (global) style
Creole on Activity

scoot Cùng "lật mở" hành trình phiêu lưu tiếp theo! Đặt ngay